EST El Kelaâ des Sraghna – Université Cadi Ayyad – Marrakech-Saf

Filière R2C – Réseaux & Cloud Computing

Bachelor – 1ère année

TRAVAUX PRATIQUES N° 2

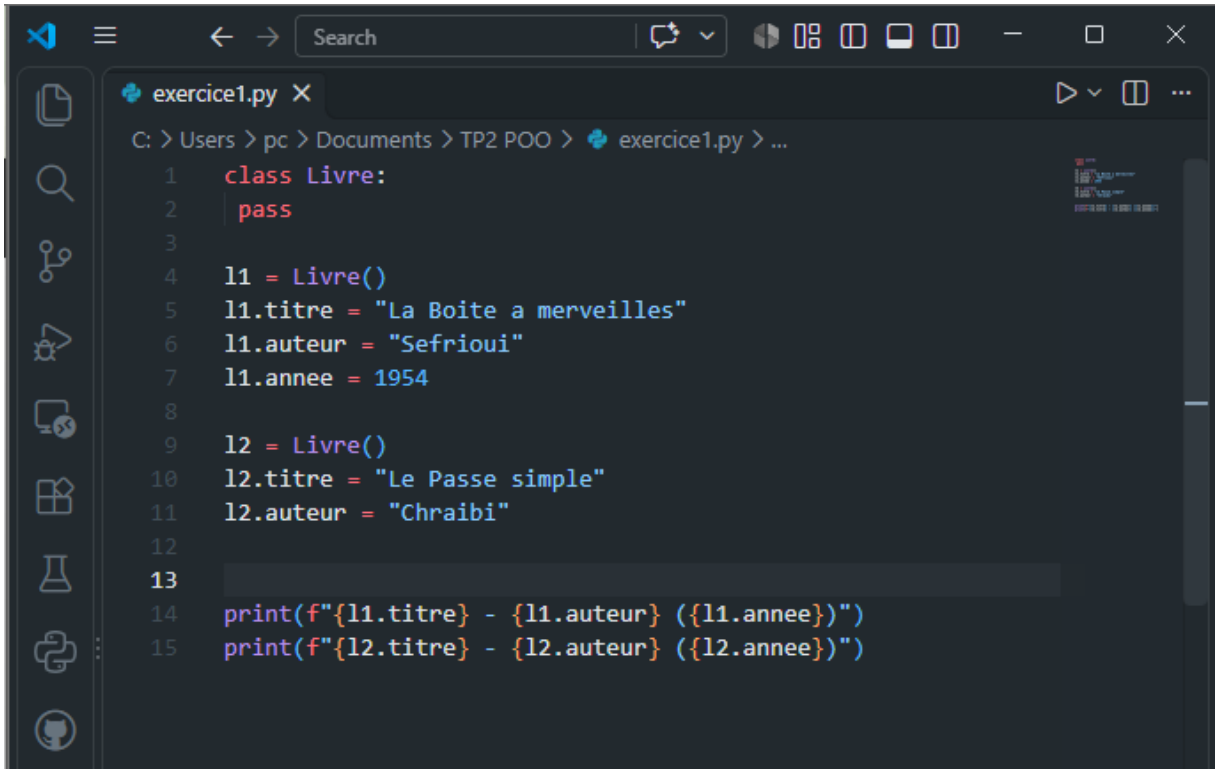
Mes premières classes en Python : les constructeurs

Nom :salma QALLALI

Année universitaire 2025–2026

Exercice 1 — Une classe sans constructeur

Étape 1 — Exécution du programme de départ :



Résultat attendu

```
PS C:\Program Files\Microsoft VS Code> & C:\Users\pc\AppData\Local\Python\pythoncore
-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice1.py"
La Boite a merveilles - Sefrioui (1954)
Traceback (most recent call last):
  File "c:\Users\pc\Documents\TP2 POO\exercice1.py", line 15, in <module>
    print(f"{l2.titre} - {l2.auteur} ({l2.annee})")
                                       ^^^^^^^^
AttributeError: 'Livre' object has no attribute 'annee'
PS C:\Program Files\Microsoft VS Code>
```

Étape 2 — Il manque l'année pour l2. Il faut ajouter :

C: > Users > pc > Documents > TP2 POO > exercice1.py > ...

```python
class Livre:
    pass

l1 = Livre()
l1.titre = "La Boite a merveilles"
l1.auteur = "Sefrioui"
l1.annee = 1954

l2 = Livre()
l2.titre = "Le Passe simple"
l2.auteur = "Chraibi"
l2.annee = 1954

print(f"{l1.titre} - {l1.auteur} ({l1.annee})")
print(f"{l2.titre} - {l2.auteur} ({l2.annee})")
```

Étape 3 — Création du troisième livre :

C: > Users > pc > Documents > TP2 POO > exercice1.py > ...

```python
class Livre:
    pass

l1 = Livre()
l1.titre = "La Boite a merveilles"
l1.auteur = "Sefrioui"
l1.annee = 1954

l2 = Livre()
l2.titre = "Le Passe simple"
l2.auteur = "Chraibi"
l2.annee = 1954

l3 = Livre()
l3.titre = "Clean Code"
l3.auteur = "Robert Martin"
l3.annee = 2008


print(f"{l1.titre} - {l1.auteur} ({l1.annee})")
print(f"{l2.titre} - {l2.auteur} ({l2.annee})")
print(f"{l3.titre} - {l3.auteur} ({l3.annee})")
```

Résultat attendu

```
PS C:\Program Files\Microsoft VS Code> & C:\Users\pc\AppData\Local\Python\pythoncore
-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice1.py"
La Boite a merveilles - Sefrioui (1954)
Le Passe simple - Chraibi (1954)
Clean Code - Robert Martin (2008)
PS C:\Program Files\Microsoft VS Code>
```

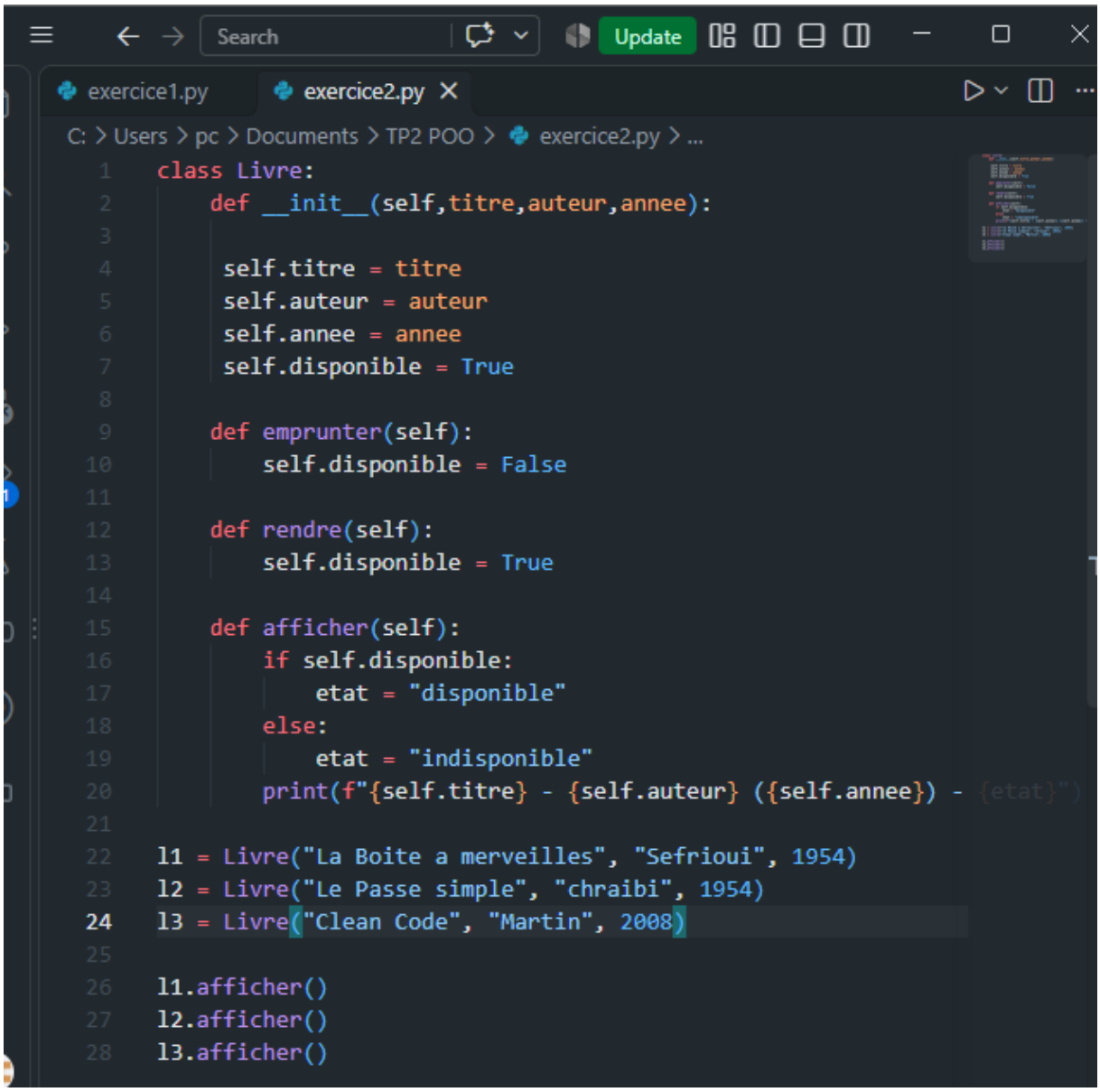
Réponses

Q1.1 — On obtient une AttributeError : 'Livre' object has no attribute 'annee'. L'erreur se produit à la **ligne 16** (la ligne print qui affiche l2.annee), car l'attribut annee n'a jamais été créé pour l2.

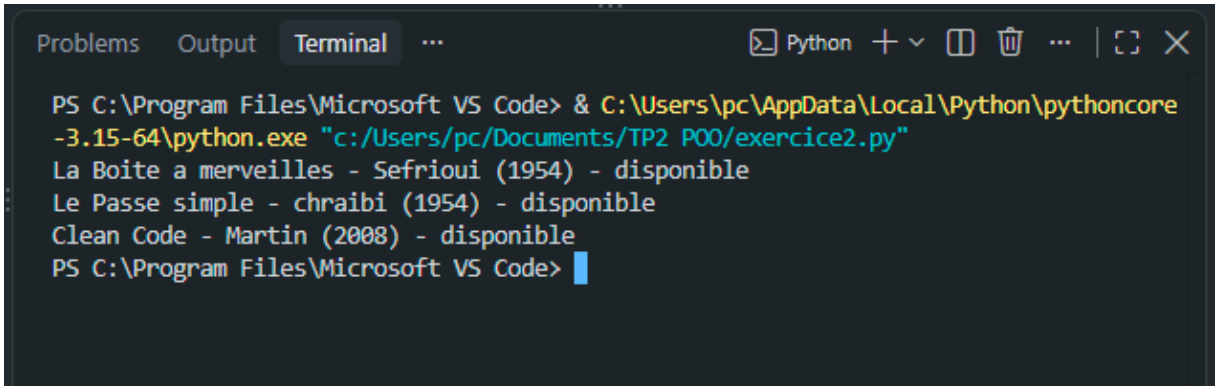
Q1.2 — Il faut écrire **4 lignes** pour créer un livre complet :

- Si l'on oublie une ligne → AttributeError au moment de l'affichage (l'attribut n'existe pas).
- Si l'on écrit l2.anee = 1954 (faute de frappe) → Python **crée un nouvel attribut** anee (sans erreur !), mais annee reste absent → AttributeError à l'affichage. C'est un danger du Python « dynamique ».

Exercice 2 – Mon premier constructeu



Résultat attendu



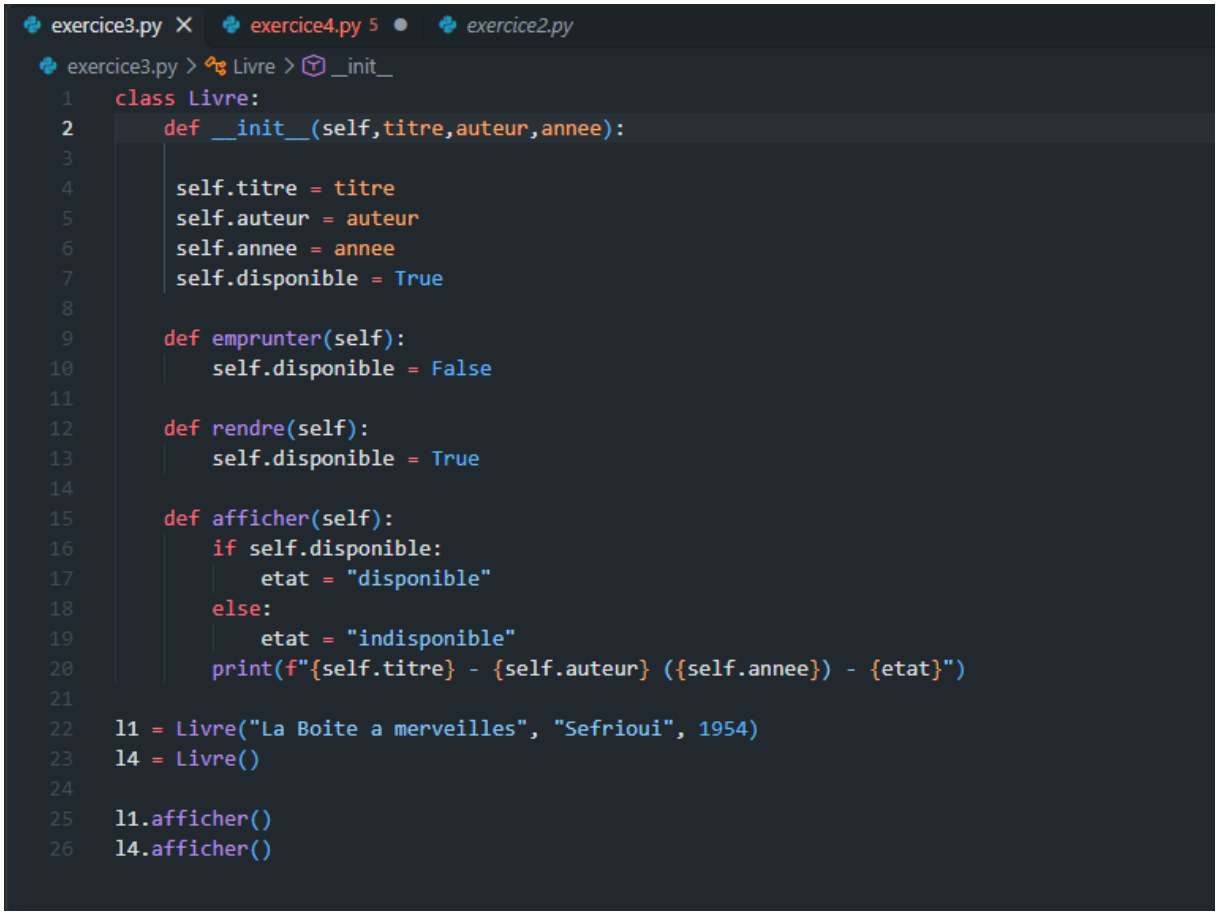
Réponses

Q2.1 — self est fourni **automatiquement par Python** : c'est la référence à l'objet en cours de création. Quand on écrit Livre(...), Python appelle en réalité Livre.__init__(nouvel_objet, ...). self n'est donc jamais passé par le programmeur.

Q2.2 —

- titre (sans self) est le **paramètre local** de la méthode : il n'existe que pendant l'exécution de __init__.
- self.titre est un **attribut de l'objet** : il persiste après l'exécution et est accessible depuis toutes les méthodes (afficher, etc.).
- self.titre = titre **copie** la valeur du paramètre local dans l'attribut de l'objet

Exercice 3 – Créer un livre sans argument



Étape 1 — l4 = Livre() provoque :

```
PS C:\Users\pc\Documents\TP2 POO> & C:\Users\pc\AppData\Local\Python\pythoncore-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice3.py"
Traceback (most recent call last):
  File "c:\Users\pc\Documents\TP2 POO\exercice3.py", line 25, in <module>
    l4 = Livre()
TypeError: Livre.__init__() missing 3 required positional arguments: 'titre', 'auteur', and 'annee'
PS C:\Users\pc\Documents\TP2 POO>
```

Étape 2 — Constructeur avec valeurs par défaut :

exercice3.py ● exercice4.py ● exercice2.py

exercice3.py > ...

```python
class Livre:
    def __init__(self, titre="Sans titre", auteur="Inconnu", annee=2026):

        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = True

    def emprunter(self):
        self.disponible = False

    def rendre(self):
        self.disponible = True

    def afficher(self):
        if self.disponible:
            etat = "disponible"
        else:
            etat = "indisponible"
        print(f"{self.titre} - {self.auteur} ({self.annee}) - {etat}")

l1 = Livre("La Boite a merveilles", "Sefrioui", 1954)
l4 = Livre()

l1.afficher()
l4.afficher()

```

Résultat attendu

```
PS C:\Users\pc\Documents\TP2 POO> & C:\Users\pc\AppData\Local\Python\pythoncore-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice3.py"
La Boite a merveilles - Sefrioui (1954) - disponible
Sans titre - Inconnu (2026) - disponible
PS C:\Users\pc\Documents\TP2 POO>
```

Réponse

Q3.1 — À l'étape 1, on obtient une TypeError car les trois paramètres du constructeur sont obligatoires : aucun argument n'est fourni alors que trois sont requis. À l'étape 3, l'erreur

disparaît car chaque paramètre possède une valeur par défaut : si l'argument n'est pas fourni, Python utilise la valeur par défaut.

Exercice 4 – Plusieurs façons de créer un livre

exercice3.py | exercice4.py | exercice2.py

exercice4.py > Livre > __init__

```python
class Livre:
    def __init__(self, titre="Sans titre", auteur="Inconnu", annee=2026):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = True

    def emprunter(self):
        self.disponible = False

    def rendre(self):
        self.disponible = True

    def afficher(self):
        if self.disponible:
            etat = "disponible"
        else:
            etat = "indisponible"
        print(f"{self.titre} - {self.auteur} ({self.annee}) - {etat}")

l1 = Livre("La Boite a merveilles", "Sefrioui", 1954)
l2 = Livre("Initiation a la POO", "Berka")
l3 = Livre()
l4 = Livre("Clean Code", annee=2008)

l1.afficher()
l2.afficher()
l3.afficher()
l4.afficher()
```

Résultat attendu

```
PS C:\Users\pc\Documents\TP2 POO>

> & C:\Users\pc\AppData\Local\Python\pythoncore-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice4.py"
La Boite a merveilles - Sefrioui (1954) - disponible
Initiation a la POO - Berka (2026) - disponible
Sans titre - Inconnu (2026) - disponible
Clean Code - Inconnu (2008) - disponible
PS C:\Users\pc\Documents\TP2 POO>
```

Réponses

Q4.1 — Pour l4, l'auteur utilisé est "Inconnu" (valeur par défaut), car il n'a pas été fourni. On écrit annee=2008 (argument nommé) parce que le 2e paramètre positionnel est auteur : écrire

simplement Livre("Clean Code", 2008) placerait 2008 dans auteur et laisserait annee à sa valeur par défaut. L'argument nommé permet de sauter un paramètre intermédiaire.

Q4.2 — Non, on ne peut pas définir deux méthodes __init__ dans une même classe Python : la seconde écrase la première (contrairement à C++/Java qui pratiquent la surcharge). En Python, on simule la surcharge par :

- **des valeurs par défaut (comme ici),**
- **des arguments nommés,**
- **ou des paramètres *args / **kwargs avec des tests dans le corps.**

Exercice 5 – Un constructeur qui refuse les erreurs

exercice3.py | exercice4.py | exercice2.py | exercice5.py

exercice5.py > Livre > afficher

```python
class Livre:
    def __init__(self,titre,auteur,annee):
        if titre == "":
            raise ValueError("le titre est vide")
        if annee < 1450 or annee > 2026:
            raise ValueError(f"annee impossible : {annee}")

        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = True

    def emprunter(self):
        self.disponible = False

    def rendre(self):
        self.disponible = True

    def afficher(self):
        if self.disponible:
            etat = "disponible"
        else:
            etat = "indisponible"
        print(f"{self.titre} - {self.auteur} ({self.annee}) - {etat}")

l1 = Livre("La Boite a merveilles", "Sefrioui", 1954)
l1.afficher()

try:
    l2 = Livre("Le Passe simple", "Chraibi", 3054)
    l2.afficher()
except ValueError as e:
    print("Livre refuse :", e)

try:
    l3 = Livre("", "Anonyme", 2000)
    l3.afficher()
except ValueError as e:
    print("Livre refuse :", e)
```

Résultat attendu



Réponses

Q5.1 — Non, la ligne l2.afficher() n'a pas été exécutée. L'exception ValueError est levée à l'intérieur du constructeur, avant même que l2 ne soit affecté. Le flux saute donc directement au bloc except, sans exécuter les lignes suivantes du try.

Q5.2 — Il est préférable de vérifier dans le constructeur car :

- ✓ **Toute création de Livre est automatiquement contrôlée (impossible de créer un livre invalide).**
- ✓ **On évite la duplication du code de vérification dans le programme principal.**
- ✓ **L'objet est garanti cohérent dès sa naissance (principe d'encapsulation).**
- ✓ **Le code de vérification est centralisé : un seul endroit à modifier si les règles changent.**

Exercice 6 – Compter les livres

exercice3.py exercice4.py exercice2.py exercice5.py exercice6.py

exercice6.py > ...

```python
class Livre:
    nombre_livres = 0

    def __init__(self, titre, auteur, annee):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = True
        Livre.nombre_livres = Livre.nombre_livres + 1
        self.numero = Livre.nombre_livres

    def afficher(self):
        print(f"#{self.numero} {self.titre} - {self.auteur} ({self.annee})")
print("Livres crees :", Livre.nombre_livres)
l1 = Livre("La Boite a merveilles", "Sefrioui", 1954)
l2 = Livre("Le Passe simple", "Chraibi", 1954)
l3 = Livre("Clean Code", "Martin", 2008)

l1.afficher()
l2.afficher()
l3.afficher()

print("Livres crees :", Livre.nombre_livres)
```

Résultat attendu

```
PS C:\Users\pc\Documents\TP2 POO> & C:\Users\pc\AppData\Local\Python\pythoncore-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice6.py"
Livres crees : 0
#1 La Boite a merveilles - Sefrioui (1954)
#2 Le Passe simple - Chraibi (1954)
#3 Clean Code - Martin (2008)
Livres crees : 3
PS C:\Users\pc\Documents\TP2 POO>
```

Réponses

Q6.1 — Il existe :

- **une valeur de** numero **par livre** (3 valeurs : 1, 2, 3) → attribut **d'instance** (self.numero).
- **une seule valeur de** nombre_livres partagée par toute la classe (3) → attribut **de classe** (Livre.nombre_livres).

Q6.2 — Si on écrit self.nombre_livres = self.nombre_livres + 1 :

- Python **lit** d'abord self.nombre_livres → ne le trouve pas dans l'instance, va chercher dans la classe → lit 0.
- Puis il **crée un attribut d'instance** self.nombre_livres valant 1 (pour chaque objet séparément).
- Livre.nombre_livres (attribut de classe) reste à **0**.

Constat : à la fin, print(Livre.nombre_livres) affiche 0, et chaque objet a son propre nombre_livres = 1. Le compteur partagé ne fonctionne plus. Il faut **rétablir** Livre.nombre_livres = Livre.nombre_livres + 1.

Exercice 7 (pour aller plus loin) – Copie ou même livre?

exercice3.py exercice4.py exercice2.py exercice5.py exercice6.py exercice7.py

exercice7.py > ...

```python
class Livre:
    def __init__(self, titre, auteur, annee):
        self.titre = titre
        self.auteur = auteur
        self.annee = annee
        self.disponible = True

    def emprunter(self):
        self.disponible = False

    def rendre(self):
        self.disponible = True

    def afficher(self):
        if self.disponible:
            etat = "disponible"
        else:
            etat = "emprunte"
        print(f"{self.titre} - {self.auteur} ({self.annee}) : {etat}")



original = Livre("Clean Code", "Martin", 2008)
meme = original
copie = Livre(original.titre, original.auteur, original.annee)

copie.emprunter()
print("Apres copie.emprunter() :")
print(" original : ", end=""); original.afficher()
print(" copie : ", end=""); copie.afficher()

meme.emprunter()
print("Apres meme.emprunter() :")
print(" original : ", end=""); original.afficher()

print(meme is original, copie is original)
```

Résultat attendu

```
Livres crees : 3
PS C:\Users\pc\Documents\TP2 POO> & C:\Users\pc\AppData\Local\Python\pythoncore-3.15-64\python.exe "c:/Users/pc/Documents/TP2 POO/exercice7.py"
Apres copie.emprunter() :
 original : Clean Code - Martin (2008) : disponible
 copie : Clean Code - Martin (2008) : emprunte
Apres meme.emprunter() :
 original : Clean Code - Martin (2008) : emprunte
True False
PS C:\Users\pc\Documents\TP2 POO>
```

Réponse

Q7.1 —

- copie = Livre(original.titre, original.auteur, original.annee) crée un **nouvel objet** (Livre(...) construit une nouvelle instance). copie et original sont donc **deux objets distincts** en mémoire : modifier l'état de copie (copie.emprunter()) ne touche pas original.

- meme = original **ne crée aucun objet** : c'est une simple **affectation de référence**. meme et original désignent **le même objet** en mémoire (d'où meme is original == True). Appeler meme.emprunter() revient donc à appeler original.emprunter() : les deux noms voient la même modification.